Project Brief: Streaming App (TanStack Query + Router)

Overview

Build a Netflix-style movie browsing app with React, TanStack Query and TanStack Router. Movie data comes from a public API (reads), and a small fake backend stores his "My List" (writes). This tests three things: **queries** (fetching and caching), **mutations** (adding, removing and rating with instant UI updates) and **routing** (pages, URL params and search params). Use a made-up app name and logo, not Netflix's. Estimated time: 4 to 5 days.

Rules

- Vite + React, with TanStack Query v5, TanStack Router (file-based routes), Tailwind and shadcn/ui. TypeScript preferred, JavaScript allowed.
- All server data goes through TanStack Query. No `useEffect` + `fetch`, and no Redux or Zustand for server data.
- Routing uses TanStack Router, not React Router.
- Keep the API token in a `.env` file and out of git.
- Every loading, error and empty state must be handled with shadcn components (`Skeleton`, `Alert`, etc.).
- Use a made-up app name and logo.

Create the project

Run these in a terminal, in order. Node 20 or newer is recommended.

1. Create the Vite app. Choose React, then TypeScript if the CLI asks.

```
npm create vite@latest streamflix -- --template react-ts
cd streamflix
npm install
```

2. Add Tailwind v4.

```
npm install tailwindcss @tailwindcss/vite
npm install -D @types/node
```

Replace everything in `src/index.css` with this one line:

```
@import "tailwindcss";
```

3. Set up the `@/` import alias. In both `tsconfig.json` and `tsconfig.app.json`, add this inside `compilerOptions` (do not add `baseUrl`):

```
"paths": { "@/*": ["./src/*"] }
```

4. Edit `vite.config.ts`. Add the Tailwind plugin, the alias and the TanStack Router plugin. The router plugin must come before `react()`.

```
import path from "node:path";
import tailwindcss from "@tailwindcss/vite";
```

```
import react from "@vitejs/plugin-react";
import { tanstackRouter } from "@tanstack/router-plugin/vite";
import { defineConfig } from "vite";

export default defineConfig({
  plugins: [
    tanstackRouter({ target: "react", autoCodeSplitting: true }),
    react(),
    tailwindcss(),
  ],
  resolve: {
    alias: { "@": path.resolve(__dirname, "./src") },
  },
});
```

5. Install shadcn/ui and the components used in this project. Accept the defaults when the init command asks questions.

```
npx shadcn@latest init
npx shadcn@latest add button card input badge skeleton alert sonner
```

6. Install TanStack Query, TanStack Router and a validation library.

```
npm install @tanstack/react-query @tanstack/react-router zod
npm install -D @tanstack/router-plugin @tanstack/eslint-plugin-query
```

7. Install the fake backend and add a script to `package.json`.

```
npm install -D json-server@0.17.4
```

```
"scripts": {
  "api": "json-server --watch db.json --port 3001 --delay 800"
}
```

8. Create `db.json` in the project root with this content:

```
{ "myList": [], "ratings": [] }
```

9. Run both servers, each in its own terminal. The app runs at `http://localhost:5173` and the fake API at `http://localhost:3001`.

```
npm run dev
npm run api
```

Create the `src/routes` folder and a `__root.tsx` file in it, then follow the TanStack Router file-based routing guide to finish wiring the router.

Get your API token

The movie data needs a free TMDB token. The My List backend (json-server) needs no key.

1. Create a free account at themoviedb.org and confirm your email.
2. Open your account **Settings**, then the **API** section, and request an API key. The form asks what you will use it for: choose personal or learning use and describe it as a practice project.
3. On the API page, copy the **API Read Access Token**. This is the long token, not the shorter API key.
4. In the project root, create a `.env` file containing `VITE_TMDB_TOKEN=<your token>` and add `.env` to `.gitignore`.

5. Test it before writing any React: `curl -H "Authorization: Bearer <your token>" https://api.themoviedb.org/3/trending/movie/week`. A JSON list of movies means it works.

Vite sends `VITE_` variables to the browser, so the token is visible there. That is fine for a practice app, but never commit it or use this setup for a real product.

Endpoints

Two APIs: TMDB for movie data (read-only) and a local fake backend for My List and ratings (read and write).

1. Movie data: TMDB (GET only)

Base URL: `https://api.themoviedb.org/3`. Create a free TMDB account, copy the **API Read Access Token** from the API settings page, and send it as the header `Authorization: Bearer <token>`. Poster images are built as `https://image.tmdb.org/t/p/w500/<poster_path>`.

Endpoint	Used for
`/trending/movie/week`	Hero banner and the "Trending now" row
`/movie/popular`	"Popular" row
`/movie/top_rated?page=1`	"Top rated" row with a Load more button
`/genre/movie/list`	Genre names and ids for the genre filter
`/discover/movie?with_genres=28&page=1`	One row per genre, and the genre page
`/movie/{id}?append_to_response=videos,credits`	Movie detail page
`/search/movie?query=batman&page=1`	Search page

2. My List and ratings: json-server (local, read and write)

Setup: `npm install -D json-server@0.17.4`, create `db.json` containing `{ "myList": [], "ratings": [] }`, then run `npx json-server --watch db.json --port 3001 --delay 800`. The 800 ms delay makes loading states and optimistic updates easy to see. Pin version 0.17.4: it is the stable release, and the v1 beta changes several options.

Method and path	Body	Used for
`GET /myList`	none	Show the My List page
`GET /myList?movieId=550`	none	Check if one movie is already saved
`POST /myList`	`{ movieId, title, posterPath }`	Add a movie

Method and path	Body	Used for
`DELETE /myList/:id`	none	Remove a movie
`GET /ratings?movieId=550`	none	Read the user's rating
`POST /ratings`	`{ movieId, value }`	Rate a movie for the first time
`PATCH /ratings/:id`	`{ value }`	Change a rating

Pages and routes

Use file-based routes. The root layout holds a navbar with the app name, a search box and links to Home and My List.

URL	Route file	What it shows	What it tests
`/`	`routes/index.tsx`	Hero banner plus Trending, Popular and Top rated rows	Several queries at once, prefetching in the loader
`/movie/$movieId`	`routes/movie.$movieId.tsx`	Poster, overview, cast, trailer, rating, Add to My List button	Path params, dependent queries
`/search?q=...&page=...`	`routes/search.tsx`	Search results with pagination	Validated search params, keeping the previous page while the next loads
`/genre/$genreId`	`routes/genre.$genreId.tsx`	Movies in one genre with a Load more button	`useInfiniteQuery`
`/my-list`	`routes/my-list.tsx`	Saved movies with a Remove button	Mutations and cache invalidation
any other URL	not-found component	A friendly 404 page	Router error handling

Requirements

Queries

- Keep all fetch functions in one `api` file. Each one checks `response.ok` and throws an error when it is false.
- Build query keys with a small factory, for example `movieKeys.detail(id)`, instead of typing arrays in components.
- Set a `staleTime` for movie data and add a one-line comment explaining the value he chose.
- Each home page row loads on its own, with its own skeleton and its own error message with a Retry button.

- The genre page uses `useInfiniteQuery` with `getNextPageParam` and a Load more button.
- Search runs only when the text has 2 or more characters (`enabled`), and uses `placeholderData: keepPreviousData` so the list does not flash empty between pages.
- Show a small "Updating..." indicator while a background refetch runs (`isFetching`).

Mutations

- **Add to My List:** `useMutation` that posts the movie, then invalidates the My List query.
- **Remove from My List:** an optimistic update. In `onMutate`, cancel running queries, save the old data and remove the movie from the cache. In `onError`, restore the saved data. In `onSettled`, invalidate the query.
- **Rating:** create the rating with POST the first time and update it with PATCH after that. Disable the stars while the request is pending.
- The button on the detail page shows the saved state of that movie (Add or Remove) from the cache.
- Show a toast on success and on error (shadcn `sonner`).

Routing

- File-based routes exactly as in the table above, with typed `Link` components and typed params.
- Put the `queryClient` in the router context. Route loaders call `queryClient.ensureQueryData(...)` so each page opens with data ready. Set `defaultPreloadStaleTime: 0` so TanStack Query decides when data is stale.
- Validate the `/search` params (`q`, `page`) with a schema and fall back to defaults when the URL is malformed. Changing the page updates the URL.
- Set `defaultPreload: 'intent'` so data loads when the user hovers a link.
- Add a `pendingComponent`, an `errorComponent` and a `notFoundComponent`.

Sources

- TMDB authentication
- json-server on GitHub
- TanStack Router: data loading
- TanStack Router: search params